Code Explanation

Field Calculator Code Explanation

The provided code is a Python module named `field_calculator.py` that implements a field calculator functionality using PySpark. It takes an input DataFrame, applies various transformations to derive new fields, and returns the resulting DataFrame with the calculated fields.

The code is designed to implement the `mplt_BNCPLS_IF23_10K_Field_Calc` mapplet functionality, which is simplified to demonstrate the calculation of a representative sample of fields. In a real-world scenario, this code would be extended to derive all 676 required fields based on specific business rules.

Overview of the Code

The `calculate_10k_fields` function takes an input DataFrame `df` and applies a series of transformations to derive new fields. The function returns the resulting DataFrame with the calculated fields. The code includes a mix of standard transformations, such as data type conversions, string formatting, and conditional logic, to derive the required fields.

Step 1: Importing Necessary Modules and Defining the Function
The code starts by importing the necessary PySpark modules, including `DataFrame` and various functions from `pyspark.sql.functions`. The `calculate_10k_fields` function is defined, which takes an input DataFrame `df` and returns the resulting DataFrame with the calculated fields.

```
from pyspark.sql import DataFrame
from pyspark.sql.functions import (
    col, lit, trim, when, substring, lpad, rpad,
    regexp_replace, upper, to_date, date_format, coalesce
)

def calculate_10k_fields(df: DataFrame) -> DataFrame:
```

Step 2: Initializing the Result DataFrame
The input DataFrame `df` is assigned to the `result_df` variable, which will be used to accumulate the calculated fields.

```
result_df = df
```

Step 3: Applying Standard Transformations for String Fields
The code applies standard transformations to derive new fields, such as standardizing the gender code, formatting the policy number, and cleaning and formatting name fields.

```
result_df = result_df.withColumn(
    "Field_1",
    when(col("BENEFICIARY_GENDER") == "M", "1")
    .when(col("BENEFICIARY_GENDER") == "F", "2")
    .otherwise("0")
)
```

```
result_df = result_df.withColumn(
    "Field_2",
    rpad(trim(col("POLICY_NUMBER")), 10, " ")
)

if "BENEFICIARY_FIRST_NAME" in df.columns:
    result_df = result_df.withColumn(
        "Field_3",
        rpad(
            regexp_replace(
                trim(upper(col("BENEFICIARY_FIRST_NAME"))),
                "[^A-Z0-9s-.]", ""
            ),
            30, " "
        )
    )
```

Step 4: Formatting Dates and Extracting Data from Parsed AURA Output
The code formats dates by converting them to the `YYYYMMDD` format and extracts data from the parsed AURA output.

```
if "APPLICATION_DATE" in df.columns:
    result_df = result_df.withColumn(
        "Field_5",
        date_format(to_date(col("APPLICATION_DATE")), "yyyyMMdd")
    )

if "PARSED_AURA_OUTPUT" in df.columns:
    result_df = result_df.withColumn(
        "Field_6",
        when(col("PARSED_AURA_OUTPUT").isNotNull(), "AURA_PRESENT")
        .otherwise("AURA_MISSING")
    )
```

Step 5: Adding Placeholder Fields for Remaining Fields
The code adds placeholder fields for the remaining fields that are not implemented in this simplified example.

```
for i in range(7, 677):
    field_name = f"Field_{i}"
    result_df = result_df.withColumn(field_name, lit(""))
```

Step 6: Returning the Result DataFrame
The final `result_df` with all the calculated fields is returned by the `calculate_10k_fields` function.

```
return result_df
```